INDEX

Name POORNIMA. S Std Sec

Roll No. Subject School/College

Sl. No.	Date	Title	Page No.	Teacher Sign/ Remarks
1.	24/9/26	Algorithms	1	
2.	8/10/26	GE	8	
3.	8/10/26	PSO.	11.	
4.	15/10/26	ACO	15.	
5	29/10/26	cuckoo searching.		

COR 8/10/26.

COR 15/10/26

COR 29/10/26

BIS
papergrid
Date: / /

24/9/26

* Summary of Algorithm
* Application of algorithm
* which fields
* versions of algorithm.
* where can it be applicable.

Algorithms.

1. Genetic Algorithm
2. particle swarm
3. Ant colony (TSP)
4. cuckoo search.
5. Grey wolf optimizer
6. parallel cellular
7. Gene Expression.

1. GENETIC ALGORITHM.

* Summary:-

- genetic algorithm (GA) is a bio-inspired optimization algorithm.
- It is inspired by natural selection and genetics.
- It is used to find a good or optimal solution to a problem.
- GA works with a population of possible solutions.
- Better solutions are given a higher chance to produce new solutions.
- The main operations are:
  - → Selection
  - → crossover
  - → mutation

Maximize $f(x) = x^2$. paperang

Date: / /

- The process is repeated for several generations
- Finally, the best solution found is selected.

Field of Applications

(1) Artificial intelligence
(2) Machine learning
(3) Robotics
(4) Engineering.
(5) Transportation.

Versions

(1) Binary GA
(2) Real-coded GA
(3) Integer-coded GA
(4) permutation GA
(5) Multi-objective GA

Advantages	disadvantages
- can solve complex optimization problems	- Does not guarantee the global optimum
- Does not require derivatives	- can require a
- Searches many solutions at the same time.	lot of computat
- Mutation helps explore new solutions.	- To much mutat can make the search random
- Flexible and can be used in many fields.	- Results can diff because of random

papergrid
Date: / /

2. Particle swarm optimization (PSO)

* Summary.

- It is bio-inspired optimization algorithm.
- It is inspired by the social behavior of birds flocking and fish schooling.
- PSO is used to find the best / optimal solution to a problem.
- instead of individuals evolving like GA, PSO uses a group of particles that move through the search spaces.

Applications.

(1) Robotics
(2) Machine learning
(3) Electrical Engineering
(4) Engineering
(5) Bioinformatics.

Advantages.	Disadvantages
- Simple to understand & implement	- Does not guarantee the global optimum
- uses relatives few mathematical operations	- can get stuck in local optimum.
- uses a population of solutions	- Results can vary because of random values.
- Does not require derivatives.	
- Has relatively few main parameters	

versions.

(1) Standard PSO
(2) Binary PSO
(3) Discrete PSO.

3. Ant - colony optimization (ACO)

- It is inspired by how ants find short paths between their nest and food.
- It is commonly used for routing and path - finding problems.
- your problem is Traveling salesman problem.
- Ants find good / short paths using pheromone trails.

versions

(1) Ant System (AS)
(2) Ant colony System (ACS)
(3) Max - min Ant System
(4) Ant colony.
(5) Rank - Based Ant System.

Applications

(1) Transportation
(2) computer Networks
(3) Scheduling
(4) Robotics
(5) Traffic Management.

Advantages	Disadvantages
- Easy to understand & implement	- can be slow for large problems
- good for routing & path problems.	- Does not guarantee
- Does not need derivatives	- parameter selection
- can explore multiple solutions	- Results can vary due to randomness

CO1
24/9/26

papergrid

Date: / /

4. cuckoo search (CS)

- Nature-inspired optimization algorithm.
- inspired by cuckoo birds laying eggs in other birds' nests.
- uses levy flights to search for new solutions
- used mainly for continuous optimization.

versions

(1) Standard cuckoo search
(2) Modified cuckoo search
(3) Binary cuckoo search
(4) Multiobjective cuckoo search.

Application.

(1) Engineering design
(2) Machine learning
(3) Data mining.

Advantages

- can avoid local minima
- good global search ability
- Few parameters

Disadvantages

- can be computationally expensive
- Doesnot guarantee the best soln
- parameter selection can affect performance

best soln can be difficult

Date: / /

5. Grey wolf optimizer (GWO).

- It is a bio-inspired swarm optimization algorithm.
- inspired by greywolves' social hierarchy and hunting behavior.
- wolves work together to find the best solution.
- mainly used for continuous optimization

versions
(1) Standard GWO
(2) Binary GWO
(3) Multi-objective GWO
(4) Adaptive GWO

Applications
(1) Engineering design
(2) Machine learning
(3) Data Analysis
(4) Feature selection

Advantages
- Simple to implement
- Few parameters
- good exploration
- works well for continuous optimization

Disadvantages
- gets stuck in local optimum
- performance depends on parameters
- can be slow for complex problems

Date: / /

6. parallel cellular Algorithms and programs.

- parallel cellular - inspired by biological cells.
- cells work simultaneously and independently.
- each cell represents a possible solution.
- cells interact with their neighboring cells.

versions.

(1) Synchronous cellular Algorithm
(2) Asynchronous cellular Algorithm.
(3) cellular GA
(4) parallel cellular Algorithm

Applications

- large - scale optimization
- Image processing.
- pattern recognition
- scheduling.

Adv

- Highly parallel
- good for large problems
- local interactions maintain diversity
- fast

Disadv

- implementation can be complex
- may not always find the global optimum
- Requires suitable parallel hardware for maximum benefit.

7. Gene Expression Algorithm (GEA)

- inspired by gene expression in living organisms
- genetic information is transformed into a functional solution
- uses selection, crossover, mutation, gene expression

version	Application
(1) Basic GEA	- Engineering design
(2) Multi-gene GEA	- Machine learning
(3) Multi objective GEA	
(4) Adaptive GEA	

Adv	Disadv.
- Does not require derivatives	- computationally exp
- can find near optimal sol	- parameter sensitivity
- flexible	- more complicated

COV
24/9/26

pseudocode for GE.

Begin
define the either function f(x)
set:
population - size
Mutation - Rate
crossover - Rate
Number of generation.

Create an initial population randomly

For generation = 1 to number of generation
// step 1:- Evaluate Fitness
For each individual in population Do
calculate fitness = f (individual)
End for

// step 2:- Select parent
select individual from population based on their fitness.
Store selected individual as parents.

// step : crossover.
Create an empty new population
while new population is not full Do
Select two parents
If random value < crossover - Rate then
perform crossover b/w the two parents
generate two offspring
Else
offspring = parents
End If

Add offspring to new_population
End while
// Step 4: Mutation
for each offspring in new population Do
If random value < mutation_Rate the
Mutate the offspring

End if
End for

// Step 5: Replace odd population
population = new_population
// Step 6: Track best solution
Find the individual with highest fitness
If fitness of current best < fitness of previous best
THEN
update best_solution
End if
End for
print best_solution
print fitness of best_solution
End

Research Summary

Travelling salesman problem optimization using genetic Algorithm.

The Travelling salesman problem (TSP) takes cities and their coordinates as input, calculates the Euclidean distance b/w cities and represents each possible tour as a chromosome.

papergrid
Date: / /

- An initial population of different tours is generated using value encoding, and each tour is evaluated based on its total travel distance, where a shorter distance means a better solution.
- Roulette wheel selection selects parents; ~~impo~~ crossover creates offspring, and interchange mutation swaps two cities to introduce variation.
- The new offspring form the next generation, and this process is repeated until convergence. Finally the chromosome with the minimum total distance is selected as the best or near-optimal route.

Output Solution :-

Best Route : [0, 1, 2, 4, 3]
Minimum distance : 19.82.

The salesman should visit the cities in the order 0 → 1 → 2 → 4 → 3 → 0.

coding completed
8/10/26

8/10/26.

(2) particle swarm optimization for Fuction optimization.

Algorithm

~~Pseudo code~~:-

Start
Define objective function $f(x)$
Initialize particle, position and velocity.
Set pbest = particle's best position
set gbest = overall best position
Repeat for each iteration:
calculate fitness
update velocity using pbest and gbest
update position
update pbest and gbest

End Repeat
print gbest and best fitness
End.

Start
$f(x) \leftarrow$ objective function
Initialize particles
Initialize position x and velocity V
For each particle
fitness $\leftarrow f(x)$
pbest $\leftarrow x$

End for
gbest $\leftarrow$ best pbest
While stopping condition is false
for each particle
calculate new velocity
$v \leftarrow$ update velocity using
pbest and gbest

calculation of V

$v = w v + c_1 r_1 (p_{best} - x) + c_2 r_2 (g_{best} - x)$

papergrid

Date: / /

update position
$x \leftarrow (x) + (v)$ → update position
New position = old position + velocity

calculate fitness
fitness ← f(x)
If fitness > pbest fitness
pbest ← x
End if
End for
gbest ← best pbest
end while
print gbest
print gbest
Print best fitness
End.

Health care

Research paper Summary:-

- "A Novel Deep learning Architecture optimization for Multiclass classification of Alzheimer's Disease level

(1) problem:- classify the level of Alzheimer's disease using MRI brain images.

(2) classes:-
- Non-demented
- very mild demented
- Mild Demented
- Moderate Demented

(3) why PSO? → To automatically find good CNN (CNN) architecture and hyperparameters instead of selecting them manually.

(4) particle:- Represents one possible CNN configuration

Date: / /

(6) position :- Represents the current CNN parameters such as numbers of layers, filters, learning rate, etc.

(7) velocity :- Determines how those CNN parameters are changed toward better configurations

(8) Fitness :- The classification performance of the CNN.

(9) pbest :- best CNN configuration found by an individual particle.

(10) gbest :- Best CNN configuration found by the entire swarm

(11) process :-

MRI Images

↓

create PSO particles

↓

each particle = CNN configuration

↓

Train CNN

↓

calculate accuracy

↓

Accuracy = Fitness

↓

update pbest & gbest

↓

update particles

↓

Repeat

↓

Best CNN architecture

↓

Alzheimer's classification

papergrid

Date: / / 0

Output Summary:

Best position :- The best solution found by pso
Best fitness:- how good that solution is
the particles start with random positions are gradually more toward the better solution.
after 20 iterations, PSO finds a value close to 8.

→ PSO searches many solution & finally finds the best position and its fitness.

coding completed

COL
8/10/26

15/10/26

Date: / Pg:

(3) Ant colony.

Start
Set cities A, B, C, D
Set distance b/w cities
Set no of ants, α, β, ρ
Set initial pheromone = 1.
for each iteration
for each ants
start with city A.
Mark A as visited
while (some cities are not visited)
calculate the closeness
eta = $\frac{1}{\text{distance}}$
calculate probability
$P = \frac{(\text{pheromone}^{\wedge}\text{alpa} * \text{eta}^{\wedge}\text{beta})}{\sum(\text{pheromone}^{\wedge}\text{alpa} * \text{eta}^{\wedge}\text{beta})}$
choose the next unvisited city.
Mark the city as visited
End while
Return to city A.
calculate the total distance D.
update the new pholomone =
(1 - rho) + added pholomone.
Save the shortest distance.
print the shortest distance
END.

TSP.

papergrid
Date: / /

Research paper Summary:-

"Using Hyper populated Ant colonies for solving the TSP"

- problem:- The paper solves the TSP - finding the shortest route that visits every city once and returns to the starting city
- Method used:- It uses Ant colony optimization (ACO), which is inspired by how ants uses pheromone trails to find good paths.
- problem with Normal ACO:- ACO can find good solutions, but its processing time can be unpredictable because ant decisions involve randomness.
- proposed solution:- The paper uses a large no of ants, called a hyper-populated ant colony
- multiple colonies:- Instead of one colony, several ant colonies can work on the same TSP problem
- parallel processing:- The colonies can work at the same time, reducing the time needed to find a good solution.
- Ant colony community (ACC):- The paper introduces a Ant colony community, where multiple colonies work together and share information.
- Main Result:- more ants and multiple colonies can reduce the number of iterations and improve processing speed while still finding good routes.
- Future use:- The approach may be especially useful for Dynamic TSP, where distances or travel conditions can change

papergrid
Date: / /

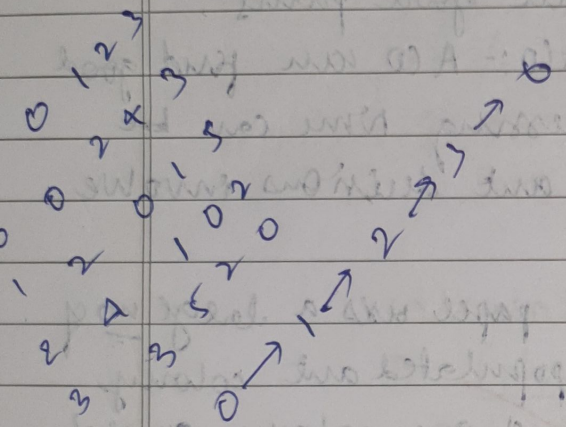
Output Summary

The ant starts from city 0
it selects cities based on distance + pheromone
the best route found 0

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$

total distance $2+1+2+3=8$

pheromone is updated after each iteration
after many iterations, ACO finds a short / good route



O/P verified

COR
15/10/25

29/10/26

papergrid

Date: / /

(4) cuckoo search. [Algorithm - ~~pseudocode~~].

Begin
Initialize nest Randomly
calculate fitness of each nest
$(100 - (x-10)^2)$
Find the best nest.
For iteration = 1 to max_iteration:
For each nest:
generate new nest = nest[i] + levy_Flight (
calculate new_fitness.
if new_fitness > fitness[i]:
nest[i] = new_nest
fitness[i] = new_fitness
End if
End for
generate new random discovery nest
calculate the fitness random discovery nest
Replace the worst nest with discovery nest
Find the current best nest:
if current best is better than best nest: current > best nest
update the best nest
End if
End for
Return best nest
END.

Date: / /

pseudocode.

```
Begin
    nests ← Random()
    fitness ← Evaluate(nests)
    best ← max(fitness)

    for i ← 1 to max_iterations
        For each nest
            new_nest ← nest + levy_flight()
            new_fitness ← Evaluate(new_nest)
            if new_fitness > fitness:
                nest ← new_nest
                fitness ← new_fitness
            End if
        End for
        discovery_nest ← Random()
        Evaluate(discovery_nest)
        Replace(worst_nests())
        best ← max(fitness)
    End for
    Return best
End.
```

Image Processing

Research paper Summary:-

paper:- "Automatic Image Enhancement Using Evolutionary optimization Algorithms" (2018).

Main objective: The paper focuses on improving the quality of images, especially images that look dark, unclear, or have low contrast.

problem:- Some images have poor brightness, unclear details or insufficient contrast, making it difficult to see objects clearly.

papergrid
Date: / /

- proposed method : the paper uses evolutionary optimization algorithms to find suitable settings for improving images.
- Algorithm used is cuckoo search :- Searches for best image enhancement settings.
- Role of cuckoo search: It generates different possible solutions and evaluates them using a fitness function. Better solutions are selected and improved over multiple iterations
- fitness function :- It measures how good an enhanced image is according to the chosen image-quality criteria. The algorithm tries to find settings that produce a better result.
- image enhancement :- The selected setting are applied to improve image contrast, visibility and details.
- main benefit :- optimization helps find suitable enhancement setting automatically instead of requiring a person to adjust every setting manually
- The paper explores how optimization algorithms can automatically find good parameters for image Quality.

Output Summary :-

- The program downloads a random colour image from the internet
- the image is used as input for the cuckoo search optimization process.
- Random nests are generated, representing different brightness and contrast settings.
- A fitness function evaluates each solution using the image's standard deviation.

- The algorithm searches for best settings over multiple iterations.
- The best brightness and contrast values are applied to the original image.
- Finally, the original and enhanced colour image are displayed side by side for comparision.

o/p verified
29/10/26

papergrid
Date: / /

5/11/26

5. Grey wolf optimizer

pseudocode:

Define objective function. F(x)
Initialize:
Number of wolves N
Maximum iterations T
lower bound LB
upper bound UB.
Generate random position
$X_i = LB + r(UB - LB)$
For t = 1 to T
calculate the fitness
Fitness = $f(x_i)$
Select three best wolves:
α = Best position
β = 2nd Best position
Delta = 3rd Best position.
calculate the coefficient:
a = 2 - 2t/T
For each wolf x_i
A1 = 2 * a * r1 → r
C1 = 2 * r2
A2 = 2 * a * r3 → r
C2 = 2 * r4
A3 = 2 * a * r5 → r
C3 = 2 * r6
D1 = |C1 * Alpha - x_i|
D2 = |C2 * Beta - x_i|
D3 = |C3 * Delta - A3 * B3|
x_1 = Alpha - A1 * D1
x_2 = Beta - A2 * D2
x_3 = Delta - A3 * D3

update positions:
$x_1 = (x1 + x2 + x3)/3$
END FOR
END FOR
print Alpha as best solution.

Research paper Summary:
"Binary Multi-objective Grey wolf optimizer for feature selection in classification (2020)"

- The paper proposes a Binary Multi-objective grey wolf optimizer for feature selection.
- It focuses on selecting important features and removing irrelevant features.
- Feature selection is treated as a multi-objective optimization problem.
- Two main objectives are considered:
  - Minimize the Number of selected features
  - Minimize classification error.
- An Artificial Neural Network (ANN) is used to evaluate the selected feature subsets.
- The method is tested using 15 benchmark datasets from the UCI repository.
- It is compared with other optimization algorithms, including MOGWO, NSGA-II and MOPSO.
- Results show improvements in feature reduction and classification error on most tested datasets.

papergrid
Date: / /

Output Summary:

- Dataset is given as input
- GWO initializes random wolves.
- Binary GWO selects important features (0 or 1)
- Fitness is calculated using 2 objectives:
- Alpha, Beta, Delta guide the search
- pareto optimization identifies the best solutions
- Final selected features are displayed
- classification accuracy is calculated.

∴

5/11